```cpp
#include <iostream>
#include <string>
using namespace std;

const int tablesize = 3;
```

۳ تا باکس بیشتر نخواستم داشته باشم چون سریع collision می فهمم و جدول ۳ تایی باعث میشه راحت تر chaining رو بفهمم.

```cpp
struct node
{
    string name;
    string type;
    node *next;
};
```

ساختار node برای نگهداری یک نماد در جدول نمادها استفاده می‌شود و سه بخش دارد: name ، type و next برای آدرس گره بعدی.مثلاً برای نماد animal از نوع string، مقدار name برابر "animal" و مقدار type برابر "string" ذخیره می‌شود. next گره‌ها را مثل زنجیر به هم وصل می‌کند: اگر گره دیگری بعد از animal در همان خانه نباشد مقدارش nullptr است، و اگر باشد آدرس آن گره در next قرار می‌گیرد.

```cpp
node *hashtable[tablesize] = {nullptr, nullptr, nullptr};

//تابع هش که هش هر اسم رو برمیگردونه
int hashfunction(string name)
{
    return name.length() % tablesize;
}
```

hashtable آرایه‌ای ۳ خانه‌ای است که هر خانه آدرس اولین گره‌ی یک لیست پیوندی را نگه می دارد . در شروع برنامه هر سه خانه nullptr هستند، یعنی هیچ نمادی ذخیره نشده است. مثلاً بعد از ذخیره‌ی animal، خانه‌ی ۰ آدرس گره‌ی animal را نگه می‌دارد.

جدول ما فقط خانه‌های ۰ و ۱ و ۲ داره. باقی‌مانده‌ی تقسیم بر ۳ **همیشه** یکی از همین سه عدده، پس شماره از جدول بیرون نمی‌زنه (اگه می‌زد، برنامه کرش می‌کرد). تابع hashfunctionنام را می‌گیرد و شماره‌ی خانه‌ی مربوطه در جدول را برمی‌گرداند.

مثال‌ها

نام	طول	محاسبه	خانه
animal	6	6 % 3 = 0	0
pencil	6	6 % 3 = 0	0

نام	طول	محاسبه	خانه
x	1	1 % 3 = 1	1
count	5	5 % 3 = 2	2

```
25    node *searchoneline(node *current, string name)
26    {
27        while (current != nullptr)
28        {
29            if (current->name == name)
30                return current;
31            current = current->next;
32        }
33        return nullptr;
34    }
```

- در اینجا یک تابع داریم به اسم searchoneline() که اشاره گر هست و اطلاعات و ادرس یک node پیدا شده رو برمیگرداند، کار این تابع اینجا این هست که اگر تابع lookup () اجرا شد، مثلا برای name = "animal" ، current node* میشود hashtable[hashfunction("animal")] ؛ هش فانکشن animal میشود صفر ، پس میره خانه صفر رو میگرده hashtable[0]=current، اگر current برابر nullptr باشد، یعنی لیست خالی است و تابع nullptr برمیگرداند .در غیر این صورت نام گره‌ی current با "animal" مقایسه می‌شود؛ اگر برابر بود، آدرس همان گره برگردانده می‌شود، وگرنه current به گره‌ی بعدی (current->next) می‌رود و این روند تکرار می‌شود تا یا گره پیدا شود یا به nullptr برسیم.

```
36    node *lookup(string name)
37    {
38        return searchoneline(hashtable[hashfunction(name)], name);
39    }
```

- تابع lookup() اشاره گر رو داریم اینجا ، در مورد این تابع قبلا صحبت کردیم ، این تابع میاد ، تابع searchoneline() رو صدا میزنه که برسی کنه ببینه ایا نام یا اسم مورد نظر وجود دارد یا ندارد. و دراخر پاسخ این تابع رو برمیگرداند .

سوال = دلیل ساخت این تابع چیه در کدت؟

```
41    bool insert(string name, string type)
42    {
43        if (lookup(name) != nullptr)
44            return false;
45
46        int index = hashfunction(name);
47        node *newnode = new node{name, type, hashtable[index]};
48        hashtable[index] = newnode;
49        return true;
50    }
```

std::__cxx11::string type

- تابع instert() که true ,false را برمیگرداند. در ابتدا شرط داریم که تابع lookup() اجرا بشه و برسی کنه وجود ندارد یا دارد .اگر تکراری باشد false را برمیکرداند . اگر nullptr باشد تابع lookup میره قسمت بعد ، تابع هش فانکشن اجرا میشود . مثلا برای animal که هش برابر است با صفراجرا میشود ، index=0 ، سپس اطلاعات نام جدید در گره جدیدی که ساخته ایم newnode ذخیره میشود و در اخر hashtable[index] = newnode; این خط **آدرس گره‌ی جدید** رو توی خانه‌ی جدول می‌ذاره تا جدول بدونه سر لیست الان کیه. اطلاعات قبلاً توی خود گره ذخیره شده.

```
52  node *deleteentry(string name)
53  {
54      int index = hashfunction(name);
55      node *current = hashtable[index];
56      node *previous = nullptr;
57
58      while (current != nullptr && current->name != name)
59      {
60          previous = current;
61          current = current->next;
62      }
63
64      if (current == nullptr)
65          return nullptr;
66
67      if (previous == nullptr)
68          hashtable[index] = current->next;
69      else
70          previous->next = current->next;
71
72      current->next = nullptr;
73      return current;
74  }
```

تابع deleteentry() گرهٔ name موردنظر را پیدا می‌کند، از linked list جدا می‌کند و آدرس آن را برمی‌گرداند. حذف واقعی حافظه (delete) بعداً در main انجام می‌شود.ابتدا hashfunction خانه‌ی جدول را حساب می‌کند. مثلاً برای animal مقدار index برابر ۰ است. سپس لیست آن خانه با دو اشاره‌گر پیمایش می‌شود: current گره‌ی فعلی و previous گره‌ی قبلی. اگر به انتهای لیست برسیم و نام پیدا نشود، nullptr برمی‌گردد.بعد از پیدا شدن گره، اگر previous برابر nullptr باشد یعنی گره سر لیست است و hashtable[index] = current -> next اجرا می‌شود. در غیر این صورت previous ->next= current->nextگره‌ی قبلی را به گره‌ی بعدی وصل می‌کند. در پایان current->next=nullptr گره را کاملاً از لیست جدا می‌کند.

لیست خانه‌ی ۰ قبل از حذف:

hashtable[0] --> [pencil] --> [animal] --> nullptr

بعد از deleteentry("animal"):

hashtable[0] --> [pencil] --> nullptr

```cpp
void display()
{
    for (int i = 0; i < tablesize; i++)
    {
        cout << "Bucket " << i << ": ";
        node *current = hashtable[i];

        if (current == nullptr)
            cout << "Empty";

        while (current != nullptr)
        {
            cout << current->name << " (" << current->type << ")";
            current = current->next;
            if (current != nullptr)
                cout << " -> ";
        }
        cout << '\n';
    }
}
```

فرض کن اینها وارد شده باشن: animal (string)، pencil (int)، x (int).

Bucket 0: pencil (int) -> animal (string)

Bucket 1: x (int)

Bucket 2: Empty

تابع display () تمام خانه‌های جدول هش را از خانه‌ی ۰ تا آخر به ترتیب چاپ می‌کند و محتوای لیست پیوندی هر خانه را نشان می‌دهد. برای هر خانه، ابتدا آدرس اولین گره در current قرار می‌گیرد. اگر current برابر nullptr باشد، کلمه‌ی Empty چاپ می‌شود. در غیر این صورت نام و نوع هر گره (مثلاً animal و string) چاپ می‌شود .

```cpp
void menu()
{
    cout << "\n===== SYMBOL TABLE =====\n"
         << "1. Insert\n2. Lookup\n"
         << "3. Delete\n4. Display\n0. Exit\n"
         << "Choose an operation: ";
}

int main()
{
    int choice = -1;
    string name, type;

    do
    {
        menu();

        if (!(cin >> choice))
        {
            cout << "Invalid input.\n";
            cin.clear();
            cin.ignore(10000, '\n');
            choice = -1;
            continue;
        }

        switch (choice)
        {
        case 1:
            cout << "Enter name and type: ";
            cin >> name >> type;
            cout << (insert(name, type) ? "Inserted successfully.\n" : "Variable already exists.\n");
            break;
        case 2:
        {
            cout << "Enter name: ";
            cin >> name;
            node *result = lookup(name);
            if (result != nullptr)
                cout << result->name << " : " << result->type << '\n';
            else
                cout << "Variable not found.\n";
            break;
        }

        case 3:
        {
            cout << "Enter name: ";
            cin >> name;
            node *removed = deleteentry(name);
            if (removed != nullptr)
            {
                cout << "Deleted: " << removed->name << '\n';
                delete removed;
            }
            else
            {
                cout << "Variable not found.\n";
            }
            break;
        }

        case 4:
            display();
            break;

        case 0:
            cout << "Exiting...\n";
            break;

        default:
            cout << "Choose a number from 0 to 4.\n";
        }
    } while (choice != 0);

    clearTable();
    return 0;
}
```

تابع menu() فقط گزینه‌های برنامه را چاپ می‌کند: افزودن، جستجو، حذف، نمایش و خروج.

در main یک حلقه‌ی do.. while منو را تکرار می‌کند تا کاربر عدد ۰ را وارد کند. شماره‌ی انتخاب‌شده در choice ذخیره می‌شود و دستور switch تابع مربوطه (insert، lookup، deleteentry یا display) را اجرا می‌کند. اگر کاربر ورودی نامعتبر (مثل حرف) وارد کند، پیام خطا چاپ می‌شود و ورودی پاک می‌شود تا برنامه قفل نکند.

در حالت حذف، گره‌ی جداشده با delete آزاد می‌شود و در پایان برنامه clearTable() همه‌ی گره‌های باقی‌مانده را آزاد می‌کند تا نشت حافظه نداشته باشیم.